

Table of Contents

1. Problem Statement	1
2. Dataset Description.....	2
3. Summary of Data Analysis	4
4. HDFS Analysis	6
5. MapReduce Analysis	11
6. Pig Analysis	15
7. Result	20
8. Conclusion	24

YouTube Video Data Analysis Using Hadoop HDFS, MapReduce and Apache Pig

1. Problem Statement

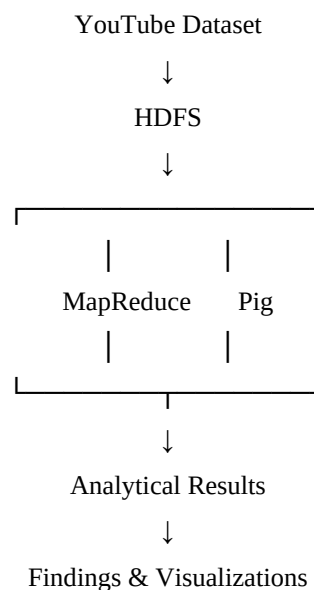
The rapid growth of online video platforms has created a large amount of data about videos, uploaders, categories, views, ratings, and comments. Processing such large datasets requires efficient storage and processing techniques.

This project analyzes a large YouTube dataset using HDFS, Java MapReduce, and Apache Pig. The main analyses are:

1. Total number of YouTube video records.
2. Number of videos in each category.
3. Total views by category.
4. Average rating by category.

The project also compares the results from MapReduce and Pig to verify the accuracy of the analysis and demonstrate different Big Data processing approaches.

The overall workflow of the project is:



Thus, the project demonstrates how a large dataset can be stored, processed, aggregated, and analyzed using Hadoop-based Big Data technologies.

2. Dataset Description

2.1 Source

The dataset used in this project is the YouTube Dataset, which contains information about YouTube videos.

The reference material identifies the original dataset source as:

YouTube Dataset:

<http://netsg.cs.sfu.ca/youtubedata/>

The dataset contains information such as video ID, uploader, age, category, length, views, rating, number of ratings, comments, and related video IDs.

For this project, the dataset was divided into five input files:

0.txt

1.txt

2.txt

3.txt

4.txt

Together, these files contain: 749,361 records

The raw dataset was used locally for preparation and was then uploaded to HDFS for Big Data processing.

2.2 Fields

The dataset contains the following major fields:

Field	Description
video_id	Unique identifier of a YouTube video
uploader	Username of the video uploader
age	Number of days between video upload date and February 15, 2007
category	Category selected for the video
length	Length of the video
views	Number of views received by the video
rate	Rating of the video
ratings	Number of ratings received
comments	Number of comments received
Related IDs	Related YouTube video IDs

The reference dataset description confirms that video_id is a unique 11-digit identifier, category represents the video category, views represents the number of views, rate represents the video rating, and ratings and comments represent the corresponding counts.

For our analysis, the most important fields were:

- category
- views
- rate

These fields were selected because they directly support category-wise video distribution, popularity, and rating analysis.

2.3 Dataset Usage in This Project

The dataset was used as the primary input for all three technologies: HDFS, MapReduce, and Pig.

First, the raw text files were uploaded to HDFS under: /bda_final_project/input

HDFS was used as the storage layer.

After that, the same HDFS data was processed using Java MapReduce and Apache Pig.

The project uses the following schema for Pig:

```
video_id:chararray
uploader:chararray
age:int
category:chararray
length:int
views:long
rate:double
ratings:int
comments:int
```

The dataset was not modified into a smaller dataset before analysis. Instead, the complete 749,361 records were processed.

The project therefore demonstrates a complete Big Data workflow from large-scale storage to data aggregation and analysis.

3. Summary of Data Analysis

3.1 Key Analytical Steps

The analysis was performed in several stages.

Step 1: Data Storage

The five YouTube dataset files were uploaded to HDFS.

The HDFS input directory was: /bda_final_project/input

The uploaded files were checked using Hadoop commands.

Step 2: HDFS Verification

After uploading the dataset, HDFS was checked to verify:

- Files were successfully uploaded.
- HDFS blocks were created.
- DataNode was active.
- No blocks were missing.
- No blocks were corrupt.
- The stored dataset was accessible.

Step 3: Total Record Analysis

The total number of records was calculated using Apache Pig.

The result was: 749,361 records

This result was also verified through HDFS data counting.

Step 4: Category-wise Video Count

The dataset was grouped according to the category field.

The number of videos in each category was then calculated.

This represents a data summarization and grouping operation.

The result showed that: Music = 179,049 videos

which was the largest category.

Step 5: Category-wise Total Views

The records were grouped by category and the views field was summed.

This analysis identifies which category accumulated the largest total number of views.

The result was: Music = 2,426,199,511 views

Step 6: Category-wise Average Rating

The records were grouped according to category and the average of the rate field was calculated.

The highest average rating was: Film & Animation ≈ 4.08

Step 7: Cross-Technology Verification

The category-wise analyses were implemented independently in:

- Java MapReduce

- Apache Pig

The results were compared.

The consistency between the two approaches provides evidence that the aggregation logic was correctly implemented.

3.2 Outputs

The project produced four major analytical outputs:

Output 1: Total Records

749,361

Output 2: Category-wise Video Count

Music → 179,049

Output 3: Category-wise Total Views

Music → 2,426,199,511

Output 4: Category-wise Average Rating

Film & Animation → 4.0777 approximately

The reference project criteria similarly include category occurrence, total views, rating-based analysis, and summarization as important analytical tasks.

3.3 Findings

The analysis produced several important findings.

Finding 1: Music is the largest category

The Music category contains 179,049 videos, which is the highest number of videos among the analyzed categories.

Finding 2: Music has the highest total views

Music also has the highest total number of views:

2,426,199,511 views

This means Music has a strong presence in both video volume and total viewing activity.

Finding 3: Film & Animation has the highest average rating

The highest average rating was found in the Film & Animation category:

approximately 4.08

Therefore, the category with the highest number of videos is not necessarily the category with the highest average rating.

Finding 4: Category-wise analysis provides useful comparison

Grouping the data by category makes it possible to compare video volume, popularity, and rating performance across different types of YouTube content.

Finding 5: MapReduce and Pig results are consistent

Both technologies produced matching analytical results for the implemented operations.

4. HDFS Analysis

Hadoop Distributed File System (HDFS) was used as the main storage layer for the YouTube Big Data Analysis project. HDFS provides a Hadoop-based file system for storing and accessing large datasets. In this project, the YouTube dataset was stored in HDFS before performing analysis using MapReduce and Apache Pig.

The Hadoop environment used for the project was:

Component	Version
Hadoop	3.2.4
Java	1.8.0_411
HDFS Project Directory	/bda_final_project

These versions were used to configure and execute the Hadoop environment for the project.

4.1 HDFS Directory Structure

A separate project directory was created in HDFS: /bda_final_project

Two subdirectories were created:

/bda_final_project/input

/bda_final_project/output

The input directory was used to store the original YouTube dataset files, while the output directory was reserved for Hadoop processing results.

The following commands were used:

```
hdfs dfs -mkdir -p /bda_final_project/input
```

```
hdfs dfs -mkdir -p /bda_final_project/output
```

These commands created the required HDFS directory structure for the project.

4.2 Uploading Dataset into HDFS

The YouTube dataset consisted of five text files:

0.txt

1.txt

2.txt

3.txt

4.txt

Each file was uploaded individually from the local dataset directory into the HDFS input directory.

The commands used were:

```
hdfs dfs -put C:\bdalfinal\bigdataset\0.txt /bda_final_project/input/
```

```
hdfs dfs -put C:\bdalfinal\bigdataset\1.txt /bda_final_project/input/
```

```
hdfs dfs -put C:\bdalfinal\bigdataset\2.txt /bda_final_project/input/
```

```
hdfs dfs -put C:\bdalfinal\bigdataset\3.txt /bda_final_project/input/
```

```
hdfs dfs -put C:\bdalfinal\bigdataset\4.txt /bda_final_project/input/
```

Therefore, the local YouTube dataset was successfully transferred into HDFS for distributed processing.

Screenshot

```
C:\bdalfinal>hdfs dfs -ls -h /bda_final_project/input
Found 5 items
-rw-r--r--  1 mdimt supergroup    54.8 K 2026-09-09 23:34 /bda_final_project/input/0.txt
-rw-r--r--  1 mdimt supergroup   889.5 K 2026-09-09 23:35 /bda_final_project/input/1.txt
-rw-r--r--  1 mdimt supergroup    6.5 M 2026-09-09 23:35 /bda_final_project/input/2.txt
-rw-r--r--  1 mdimt supergroup   56.8 M 2026-09-09 23:35 /bda_final_project/input/3.txt
-rw-r--r--  1 mdimt supergroup  139.6 M 2026-09-09 23:35 /bda_final_project/input/4.txt
```

Fig1: hdfs_upload.png.

Description

The screenshot shows the successful uploading of the five YouTube dataset files into the HDFS input directory.

4.3 HDFS Block Analysis

One of the important features of HDFS is that large files are divided into blocks for storage and processing.

The block information for the project dataset was checked using:

```
hdfs fsck /bda_final_project/input -files -blocks -locations
```

This command provides information about:

- HDFS files
- Number of blocks
- Block locations
- Replication
- File-system health

The analysis showed that the five input files occupied a total of: 6 HDFS blocks

The distribution was:

File	HDFS Blocks
0.txt	1
1.txt	1
2.txt	1
3.txt	1
4.txt	2
Total	6

The largest file, 4.txt, required two HDFS blocks, while each of the other four files occupied one block. This demonstrates how HDFS manages files by dividing them into blocks rather than treating the entire dataset as one storage unit.

Screenshot

```
/bda_final_project/input/4.txt 146345957 bytes, replicated: replication=1, 2 block(s): OK
0. BP-463196398-192.168.1.105-1785090964085:blk_1073741909_1085 len=134217728 Live_repl=1 [DatanodeInfoWithStorage[127.0.0.1:9866,DS-93d006e2-c07f-4021-a5ff-48a9065ea91d,DISK]]
1. BP-463196398-192.168.1.105-1785090964085:blk_1073741910_1086 len=12128229 Live_repl=1 [DatanodeInfoWithStorage[127.0.0.1:9866,DS-93d006e2-c07f-4021-a5ff-48a9065ea91d,DISK]]

Status: HEALTHY
Number of data-nodes: 1
Number of racks: 1
Total dirs: 1
Total symlinks: 0

Replicated Blocks:
Total size: 213620517 B
Total files: 5
Total blocks (validated): 6 (avg. block size 35603419 B)
Minimally replicated blocks: 6 (100.0 %)
Over-replicated blocks: 0 (0.0 %)
Under-replicated blocks: 0 (0.0 %)
Mis-replicated blocks: 0 (0.0 %)
```

Fig2: HDFS file, block by the hdfs fsck command.

```
Number of data-nodes: 1
Number of racks: 1
Total dirs: 1
Total symlinks: 0

Replicated Blocks:
Total size: 213620517 B
Total files: 5
Total blocks (validated): 6 (avg. block size 35603419 B)
Minimally replicated blocks: 6 (100.0 %)
Over-replicated blocks: 0 (0.0 %)
Under-replicated blocks: 0 (0.0 %)
Mis-replicated blocks: 0 (0.0 %)
Default replication factor: 1
Average block replication: 1.0
Missing blocks: 0
Corrupt blocks: 0
Missing replicas: 0 (0.0 %)

Erasure Coded Block Groups:
Total size: 0 B
Total files: 0
Total block groups (validated): 0
Minimally erasure-coded block groups: 0
Over-erasure-coded block groups: 0
Under-erasure-coded block groups: 0
Unsatisfactory placement block groups: 0
Average block group size: 0.0
Missing block groups: 0
Corrupt block groups: 0
Missing internal blocks: 0
FSCK ended at Wed Sep 09 23:58:52 BDT 2026 in 13 milliseconds

The filesystem under path '/bda_final_project/input' is HEALTHY
C:\bdalfinal>_
```

Fig3: HDFS block-location information generated by the hdfs fsck command.

Description: The screenshots show the HDFS file, block, and block-location information generated by the hdfs fsck command.

4.4 DataNode Analysis

The Hadoop DataNode status was checked using:

```
hdfs dfsadmin -report
```

This command provides information about the available DataNodes and their storage capacity.

The project environment successfully reported an active DataNode.

The DataNode report was used to verify that Hadoop had an active storage node available for storing and processing the project dataset.

Screenshot:

```
C:\bdalfinal>hdfs dfsadmin -report
Configured Capacity: 127445282816 (118.69 GB)
Present Capacity: 32116406321 (29.91 GB)
DFS Remaining: 31842705408 (29.66 GB)
DFS Used: 273700913 (261.02 MB)
DFS Used%: 0.85%
Replicated Blocks:
    Under replicated blocks: 10
    Blocks with corrupt replicas: 0
    Missing blocks: 0
    Missing blocks (with replication factor 1): 0
    Low redundancy blocks with highest priority to recover: 10
    Pending deletion blocks: 0
Erasure Coded Block Groups:
    Low redundancy block groups: 0
    Block groups with corrupt internal blocks: 0
    Missing block groups: 0
    Low redundancy blocks with highest priority to recover: 0
    Pending deletion blocks: 0

-----
Live datanodes (1):

Name: 127.0.0.1:9866 (127.0.0.1)
Hostname: DESKTOP-MJE7U90.netis.cc
Decommission Status : Normal
Configured Capacity: 127445282816 (118.69 GB)
DFS Used: 273700913 (261.02 MB)
Non DFS Used: 95328942031 (88.78 GB)
DFS Remaining: 31842639872 (29.66 GB)
DFS Used%: 0.21%
DFS Remaining%: 24.99%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 1
Last contact: Thu Sep 10 00:01:51 BDT 2026
Last Block Report: Wed Sep 09 23:27:27 BDT 2026
Num of Blocks: 49
```

Fig4: The active DataNode and its available storage information.

Description: The screenshot shows the Hadoop DataNode report, including the active DataNode and its available storage information.

4.5 Record Count Verification

The total number of records stored in the HDFS input files was independently verified using:

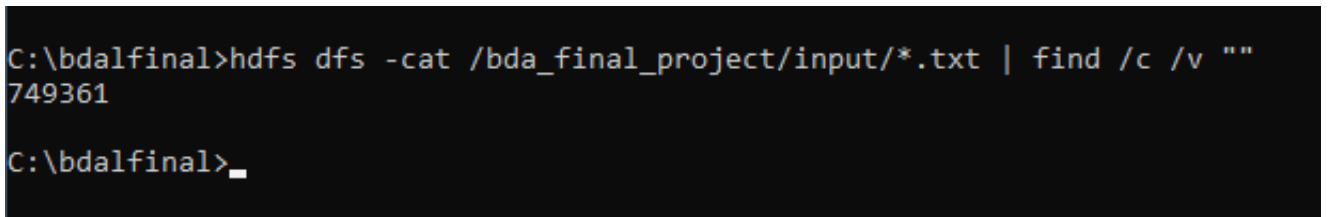
```
hdfs dfs -cat /bda_final_project/input/*.txt | find /c /v ""
```

The result was: 749,361

Therefore: Total records stored in HDFS = 749,361

This verification is important because it confirms that the complete dataset was available in HDFS before performing MapReduce and Pig analysis.

Screenshot



```
C:\bdalfinal>hdfs dfs -cat /bda_final_project/input/*.txt | find /c /v ""
749361
C:\bdalfinal>_
```

Fig5: final_varification.png.

Description

The screenshot shows the final HDFS record-count verification and confirms that 749,361 records were available from the HDFS input directory.

4.6 HDFS Health and Final Verification

After uploading and checking the dataset, the HDFS input directory was verified for block health.

The final HDFS results were:

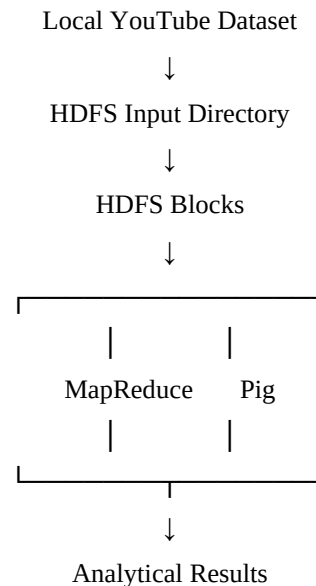
HDFS Property	Result
Dataset Files	5
Total Records	749,361
Total HDFS Blocks	6
Replication Factor	1
Missing Blocks	0
Corrupt Blocks	0
Under-replicated Blocks	0
HDFS Status	Successfully uploaded and verified

These results confirm that the dataset was successfully stored in HDFS and was ready for subsequent MapReduce and Pig processing.

4.7 Role of HDFS in the Project

HDFS acted as the foundation of the Big Data processing workflow.

The complete storage and processing flow was:



HDFS was responsible for storing the large dataset and making it available to the Hadoop processing framework. MapReduce and Pig then accessed the HDFS data to perform category-wise counting, total-view calculation, and average-rating analysis.

Thus, HDFS completed the storage and data-management stage of the project before the analytical processing stages.

5. MapReduce Analysis

Java MapReduce was used to perform category-based analytical operations on the YouTube dataset.

The project implemented three MapReduce programs:

CategoryCount.java

CategoryTotalViews.java

CategoryAverageRating.java

MapReduce follows a general processing pattern in which the Mapper extracts and transforms input data, while the Reducer aggregates values belonging to the same key. The reference material also describes Driver, Mapper, and Reducer roles for analytical jobs.

5.1 Category-wise Video Count

Objective

The purpose of CategoryCount.java is to determine the total number of videos belonging to each YouTube category.

Processing

The Mapper reads each input record and extracts the category field.

It produces a key-value pair:

Category → 1

For example:

Music → 1

Music → 1

Music → 1

Comedy → 1

During the Reduce stage, values belonging to the same category are added:

Music → 179049

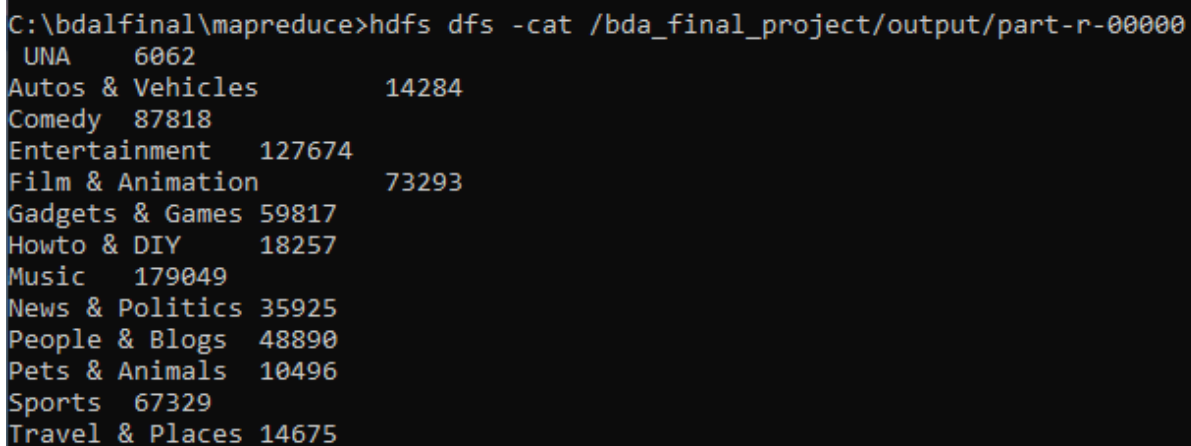
Comedy → 87818

Result

The analysis produced: Music → 179,049 videos

This was the largest category.

Screenshot



```
C:\bdalfinal\mapreduce>hdfs dfs -cat /bda_final_project/output/part-r-00000
UNA      6062
Autos & Vehicles      14284
Comedy    87818
Entertainment  127674
Film & Animation      73293
Gadgets & Games  59817
Howto & DIY      18257
Music     179049
News & Politics  35925
People & Blogs   48890
Pets & Animals  10496
Sports    67329
Travel & Places  14675
```

Fig6: category_count_result.png.

Description

The screenshot shows the successful MapReduce execution and the final category-wise video counts generated by the Hadoop job.

5.2 Category-wise Total Views

Objective

The CategoryTotalViews.java program calculates the total number of views accumulated by each category.

Processing

The Mapper extracts:

Category → Views

For example:

Music → 500

Music → 1200

Music → 3000

The Reducer then performs the summation:

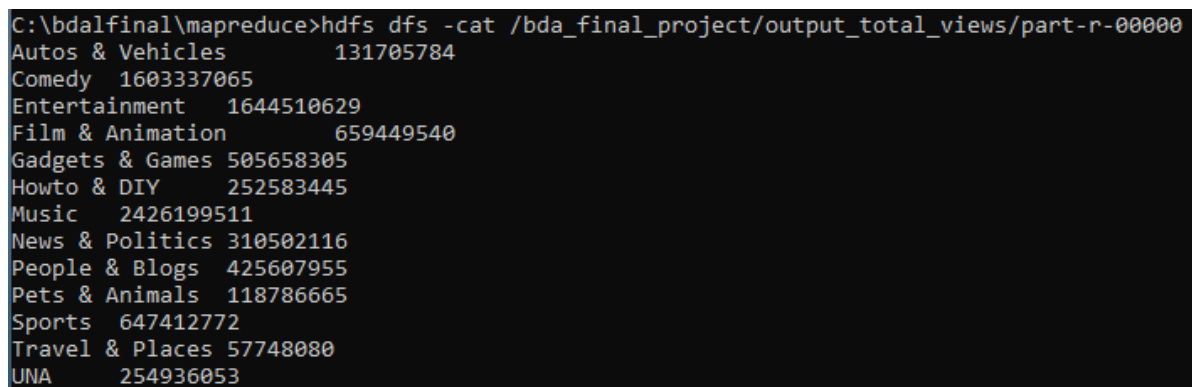
Music → 4,700

This process is repeated for every category.

Result

The highest total number of views was:

Music → 2,426,199,511 views



```
C:\bdalfinal\mapreduce>hdfs dfs -cat /bda_final_project/output_total_views/part-r-00000
Autos & Vehicles      131705784
Comedy 1603337065
Entertainment 1644510629
Film & Animation      659449540
Gadgets & Games 505658305
Howto & DIY 252583445
Music 2426199511
News & Politics 310502116
People & Blogs 425607955
Pets & Animals 118786665
Sports 647412772
Travel & Places 57748080
UNA 254936053
```

Screenshot

Fig7:category_total_views_result.png.

Description

The screenshot shows the output of the MapReduce job where the total views for each YouTube category are displayed.

5.3 Category-wise Average Rating

Objective

The CategoryAverageRating.java program calculates the average rating for videos belonging to each category.

Processing

The Mapper extracts:

Category → Rating

The Reducer aggregates the rating values and calculates the category-level average.

Conceptually:

Average Rating =

Sum of Ratings / Number of Videos

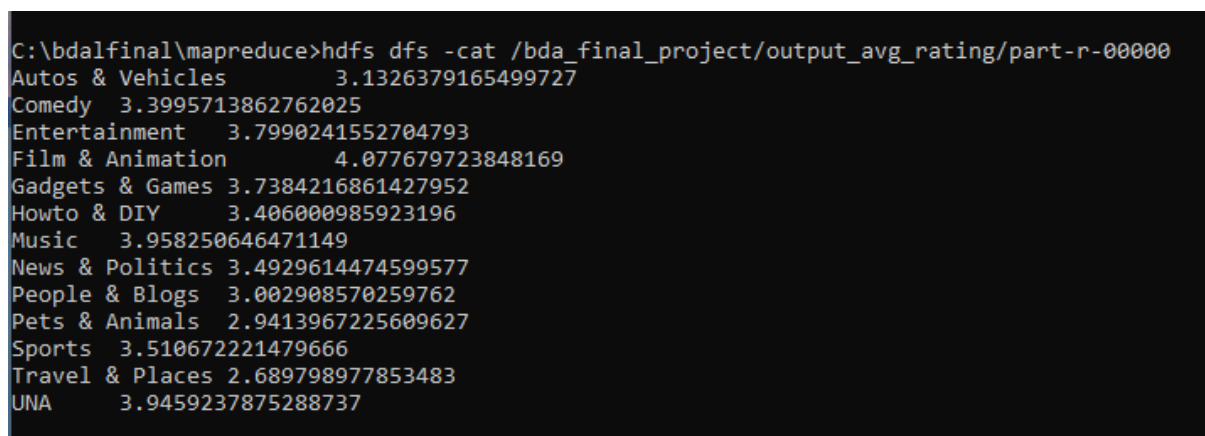
The operation is performed separately for each category.

Result

The highest average rating was:

Film & Animation → approximately 4.08

Screenshot



```
C:\bdalfinal\mapreduce>hdfs dfs -cat /bda_final_project/output_avg_rating/part-r-00000
Autos & Vehicles      3.1326379165499727
Comedy  3.3995713862762025
Entertainment  3.7990241552704793
Film & Animation      4.077679723848169
Gadgets & Games 3.7384216861427952
Howto & DIY    3.406000985923196
Music    3.958250646471149
News & Politics 3.4929614474599577
People & Blogs  3.002908570259762
Pets & Animals  2.9413967225609627
Sports   3.510672221479666
Travel & Places 2.689798977853483
UNA      3.9459237875288737
```

Fig8: category_average_rating_result.png.

Screenshot Description

The screenshot displays the MapReduce output containing the average rating calculated for each YouTube category.

5.4 MapReduce Job Execution

All three MapReduce jobs were executed successfully.

The first category-count job processed: 749,361 input records

The MapReduce processing successfully generated aggregated category-level results.

The category count analysis produced 743,569 valid categorized records, while 5,792 records had empty/missing category values. These records were visible as an empty/blank category in the Pig grouping result and were treated as missing category values rather than being silently assigned to another category.

This is an important data-quality observation because Big Data analysis can contain incomplete or missing values.

5.5 MapReduce Analysis Summary

MapReduce Program	Purpose	Main Result
CategoryCount.java	Count videos by category	Music = 179,049
CategoryTotalViews.java	Sum views by category	Music = 2,426,199,511
CategoryAverageRating.java	Average rating by category	Film & Animation $\approx$ 4.08

The implemented operations represent grouping and summarization, which are key concepts highlighted in the reference criteria.

6. Pig Analysis

Apache Pig was used as a high-level data processing tool to perform the same type of analysis using Pig Latin.

Pig was selected because it provides a simpler data-flow approach for operations such as:

- Loading data
- Filtering data
- Grouping records
- Counting records
- Summing values
- Calculating averages
- Displaying results

The reference criteria also include several Pig-based ranking and category analyses, such as top categories, top-rated videos, and top-viewed videos. In our implementation, the focus was instead on record counting, category count, category total views, and category average rating.

6.1 Loading the Dataset

The dataset was loaded from HDFS using: `/bda_final_project/input/*.txt`

The data was read using:

```
PigStorage('t')
```

A schema was defined for the input fields:

```
video_id
uploader
age
category
length
views
rate
ratings
comments
```

This allowed Pig to interpret the fields according to their appropriate data types.

6.2 Total Record Count

The `count_data.pig` script was used to determine the total number of records.

The data was grouped as a single group and the number of records was calculated.

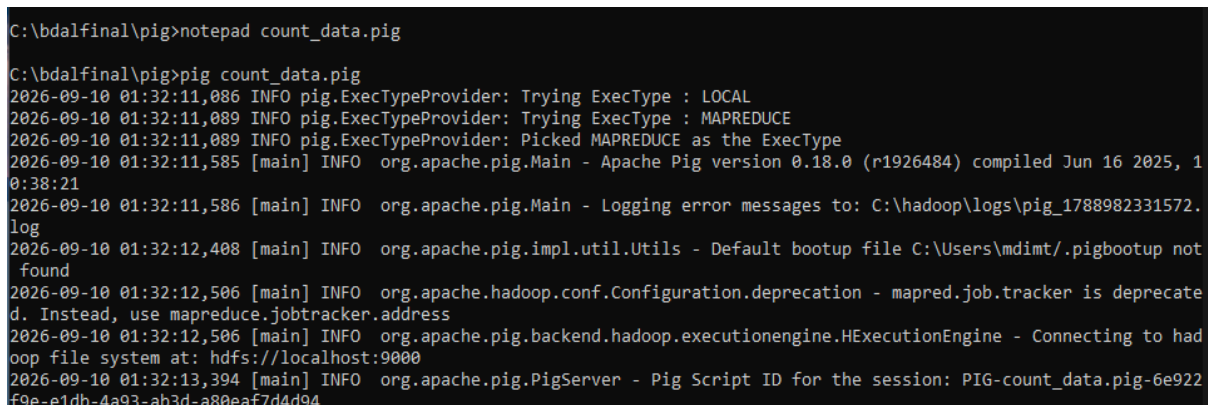
The output was:

749361

Therefore:

Total YouTube records = 749,361

Screenshot



```
C:\bdalfinal\pig>notepad count_data.pig
C:\bdalfinal\pig>pig count_data.pig
2026-09-10 01:32:11,086 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2026-09-10 01:32:11,089 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE
2026-09-10 01:32:11,089 INFO pig.ExecTypeProvider: Picked MAPREDUCE as the ExecType
2026-09-10 01:32:11,585 [main] INFO org.apache.pig.Main - Apache Pig version 0.18.0 (r1926484) compiled Jun 16 2025, 10:38:21
2026-09-10 01:32:11,586 [main] INFO org.apache.pig.Main - Logging error messages to: C:\hadoop\logs\pig_1788982331572.log
2026-09-10 01:32:12,408 [main] INFO org.apache.pig.impl.util.Utils - Default bootup file C:\Users\mdimt/.pigbootup not found
2026-09-10 01:32:12,506 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-09-10 01:32:12,506 [main] INFO org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to hadoop file system at: hdfs://localhost:9000
2026-09-10 01:32:13,394 [main] INFO org.apache.pig.PigServer - Pig Script ID for the session: PIG-count_data.pig-6e922f9e-e1db-4a93-ab3d-a80eaf7d4d94
749361
```

Fig9: *pig_count_result_1.png*

```

2026-09-10 01:41:08,976 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020.
Already tried 8 time(s); retry policy is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-10 01:41:12,025 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020.
Already tried 9 time(s); retry policy is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-10 01:41:14,173 [main] WARN org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Unable to retrieve job to compute warning aggregation.
2026-09-10 01:41:14,174 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Success!
2026-09-10 01:41:14,195 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - yarn.resourcemanager.system-metrics-publisher.enabled is deprecated. Instead, use yarn.system-metrics-publisher.enabled
2026-09-10 01:41:14,199 [main] INFO org.apache.pig.data.SchemaTupleBackend - Key [pig.schematuple] was not set... will not generate code.
2026-09-10 01:41:14,231 [main] INFO org.apache.hadoop.mapreduce.lib.input.FileInputFormat - Total input files to process : 1
2026-09-10 01:41:14,232 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths to process : 1
(749361)
2026-09-10 01:41:14,415 [main] INFO org.apache.pig.Main - Pig script completed in 9 minutes, 3 seconds and 419 milliseconds (543419 ms)

```

Fig10: pig_count_result_2.png here.

Description

The screenshots show the execution of the Pig script and the resulting total record count.

6.3 Category-wise Video Count

The category_count.pig script was used for category-wise counting.

The data was grouped using:

GROUP data BY category

Then:

COUNT(data)

was applied to determine the number of videos in each category.

Result

The highest category was: Music → 179,049 videos

The Pig output also showed records with an empty category, corresponding to missing category values.

Screenshot

```

C:\bdalfinal\pig>notepad category_count.pig
C:\bdalfinal\pig>pig category_count.pig
2026-09-10 01:43:44,151 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2026-09-10 01:43:44,154 INFO pig.ExecTypeProvider: Trying ExecType as MAPREDUCE
2026-09-10 01:43:44,154 INFO pig.ExecTypeProvider: Picked MAPREDUCE as the ExecType
2026-09-10 01:43:44,574 [main] INFO org.apache.pig.Main - Apache Pig version 0.18.0 (r1926484) compiled Jun 16 2025, 10:38:21
2026-09-10 01:43:44,575 [main] INFO org.apache.pig.Main - Logging error messages to: C:\hadoop\logs\pig_1788983024564.log
2026-09-10 01:43:45,370 [main] INFO org.apache.pig.impl.util.Utils - Default bootup file C:\Users\mdimt/.pigbootup not found
2026-09-10 01:43:45,456 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-09-10 01:43:45,456 [main] INFO org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to hadoop file system at: hdfs://localhost:9000

```

Fig11: pig_category_count_result_1.png

```

ss : 1
2026-09-10 01:52:41,872 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths
to process : 1
(UNA,6062)
(Music,179049)
(Comedy,87818)
(Sports,67329)
(Howto & DIY,18257)
(Entertainment,127674)
(People & Blogs,48890)
(Pets & Animals,10496)
(Gadgets & Games,59817)
(News & Politics,35925)
(Travel & Places,14675)
(Autos & Vehicles,14284)
(Film & Animation,73293)
(,5792)
2026-09-10 01:52:42,055 [main] INFO org.apache.pig.Main - Pig script completed in 8 minutes, 57 seconds and 998 millis
econds (537998 ms)

```

Fig12:pig_category_count_result_2.png.

Description

The screenshots show the category grouping and the final count generated by Pig.

6.4 Category-wise Total Views

The category_total_views.pig script was used to calculate total views by category.

The dataset was first grouped:

GROUP data BY category

Then the views were summed:

SUM(data.views)

This produced the total views for each category.

Result

The highest total views were:

Music → 2,426,199,511 views

Screenshot

```

C:\bdalfinal\pig>pig category_total_views.pig
2026-09-10 01:54:58,896 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2026-09-10 01:54:58,900 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE
2026-09-10 01:54:58,900 INFO pig.ExecTypeProvider: Picked MAPREDUCE as the ExecType
2026-09-10 01:54:59,356 [main] INFO org.apache.pig.Main - Apache Pig version 0.18.0 (r1926484) compiled Jun 16 2025, 1
0:38:21
2026-09-10 01:54:59,356 [main] INFO org.apache.pig.Main - Logging error messages to: C:\hadoop\logs\pig_1788983699346.
log
2026-09-10 01:55:00,262 [main] INFO org.apache.pig.impl.util.Utils - Default bootup file C:\Users\mdimt/.pigbootup not
found
2026-09-10 01:55:00,355 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecate
d. Instead, use mapreduce.jobtracker.address
2026-09-10 01:55:00,355 [main] INFO org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to had
oop file system at: hdfs://localhost:9000
2026-09-10 01:55:01,760 [main] INFO org.apache.pig.Main - Pig script ID for the current PIG script: total_views

```

Fig13:pig_category_total_views_result_1.png

```

ss : 1
2026-09-10 02:03:56,406 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths
to process : 1
(UNA,254936053)
(Music,2426199511)
(Comedy,1603337065)
(Sports,647412772)
(Howto & DIY,252583445)
(Entertainment,1644510629)
(People & Blogs,425607955)
(Pets & Animals,118786665)
(Gadgets & Games,505658305)
(News & Politics,310502116)
(Travel & Places,57748080)
(Autos & Vehicles,131705784)
(Film & Animation,659449540)
(,)
2026-09-10 02:03:56,597 [main] INFO org.apache.pig.Main - Pig script completed in 8 minutes, 57 seconds and 792 millis
seconds (537792 ms)

```

Fig14: pig_category_total_views_result_2.png.

Description

The screenshots show the successful execution of the Pig total-view analysis and the category-wise aggregated output.

6.5 Category-wise Average Rating

The category_average_rating.pig script was used to calculate the average rating by category.

The data was grouped by:

category

and the average was calculated using:

AVG(data.rate)

Result

The highest average rating was:

Film & Animation → approximately 4.08

Screenshot

```

not generate code.
2026-09-10 02:14:19,318 [main] INFO org.apache.hadoop.mapreduce.lib.input.FileInputFormat - Total input fi
ss : 1
2026-09-10 02:14:19,319 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total
to process : 1
(UNA,3.945923787528868)
(Music,3.9582506464710625)
(Comedy,3.39957138627618)
(Sports,3.5106722214796053)
(Howto & DIY,3.4060009859232054)
(Entertainment,3.799024155270472)
(People & Blogs,3.00290857025975)
(Pets & Animals,2.9413967225609725)
(Gadgets & Games,3.738421686142738)
(News & Politics,3.492961447459976)
(Travel & Places,2.6897989778534828)
(Autos & Vehicles,3.13263791654998)
(Film & Animation,4.077679723848124)
(,)
2026-09-10 02:14:19,526 [main] INFO org.apache.pig.Main - Pig script completed in 8 minutes, 57 seconds an
econds (537486 ms)

```

Fig15:pig_category_average_rating_result.png.

Description

The screenshots show the execution of the Pig average-rating analysis and the resulting category-wise average ratings.

6.6 Pig Analysis Summary

Pig Script	Analysis	Main Result
count_data.pig	Total record count	749,361
category_count.pig	Videos by category	Music = 179,049
category_total_views.pig	Total views by category	Music = 2,426,199,511
category_average_rating.pig	Average rating by category	Film & Animation $\approx$ 4.08

The Pig implementation provides a higher-level way to express the same aggregation operations that were implemented using Java MapReduce.

7. Result

The project successfully completed the required Big Data processing workflow using HDFS, MapReduce, and Apache Pig.

7.1 Overall Dataset Result

The complete dataset contained: 749,361 records

The records were stored in HDFS and successfully accessed for processing.

7.2 HDFS Result

HDFS successfully stored the dataset.

The final verification showed:

HDFS Property	Result
Total Records	749,361
Total Blocks	6
Replication Factor	1
Missing Blocks	0
Corrupt Blocks	0
Under-replicated Blocks	0

Therefore, the HDFS storage stage was completed successfully.

7.3 MapReduce Result

Three MapReduce analytical programs were successfully executed.

Category Count

Music = 179,049 videos

Category Total Views

Music = 2,426,199,511 views

Category Average Rating

Film & Animation $\approx$ 4.08

7.4 Pig Result

The same major analytical concepts were implemented using Pig.

The results were consistent with MapReduce:

Analysis	MapReduce	Pig
Total Records	749,361	749,361
Music Video Count	179,049	179,049
Music Total Views	2,426,199,511	2,426,199,511
Highest Average Rating	Film & Animation $\approx$ 4.08	Film & Animation $\approx$ 4.08

This consistency demonstrates that the analytical logic was correctly implemented in both technologies.

7.5 Main Analytical Findings

1. Most represented category

Music had the highest number of videos: 179,049 videos

2. Most viewed category

Music had the highest total number of views: 2,426,199,511 views

3. Highest-rated category

Film & Animation had the highest average rating: approximately 4.08

4. Missing category data

A total of 5,792 records had empty/missing category values. These were visible separately in the category aggregation results.

This demonstrates the importance of considering data quality during Big Data analysis.

7.6 Visualization Results

Three visualizations were generated from the analytical results.

Category-wise Video Count

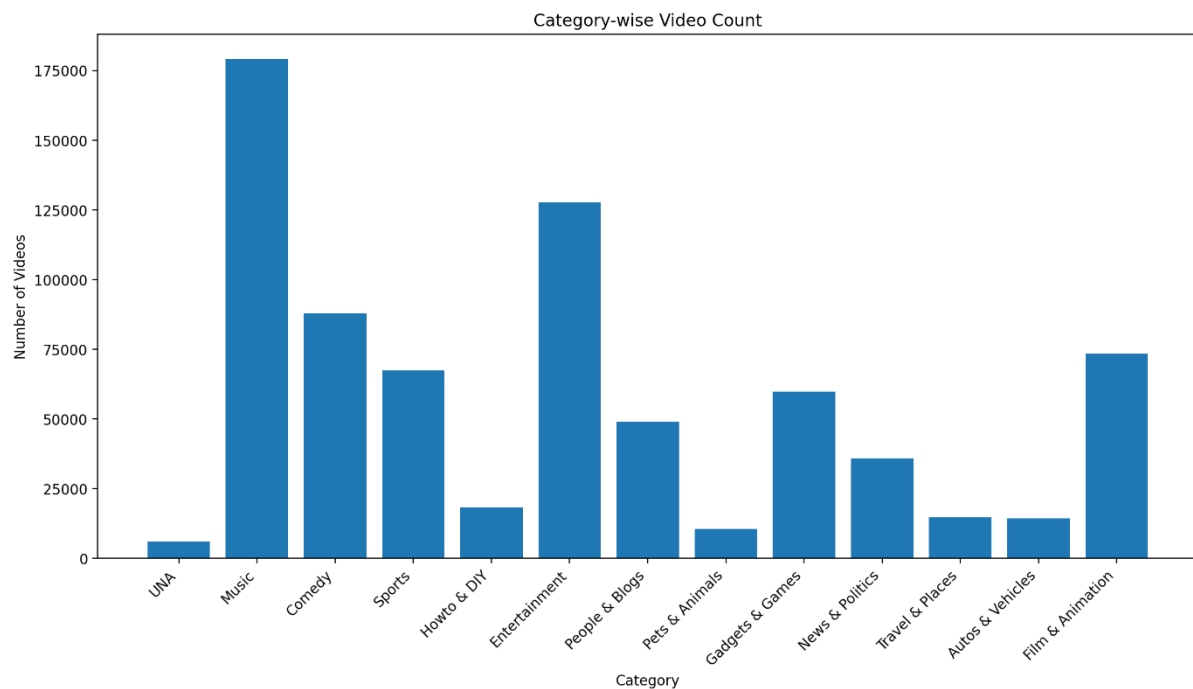


Fig16: category_video_count.png.

Description:

This visualization compares the number of YouTube videos across categories. Music has the highest number of videos, followed by Entertainment and Comedy.

Category-wise Total Views

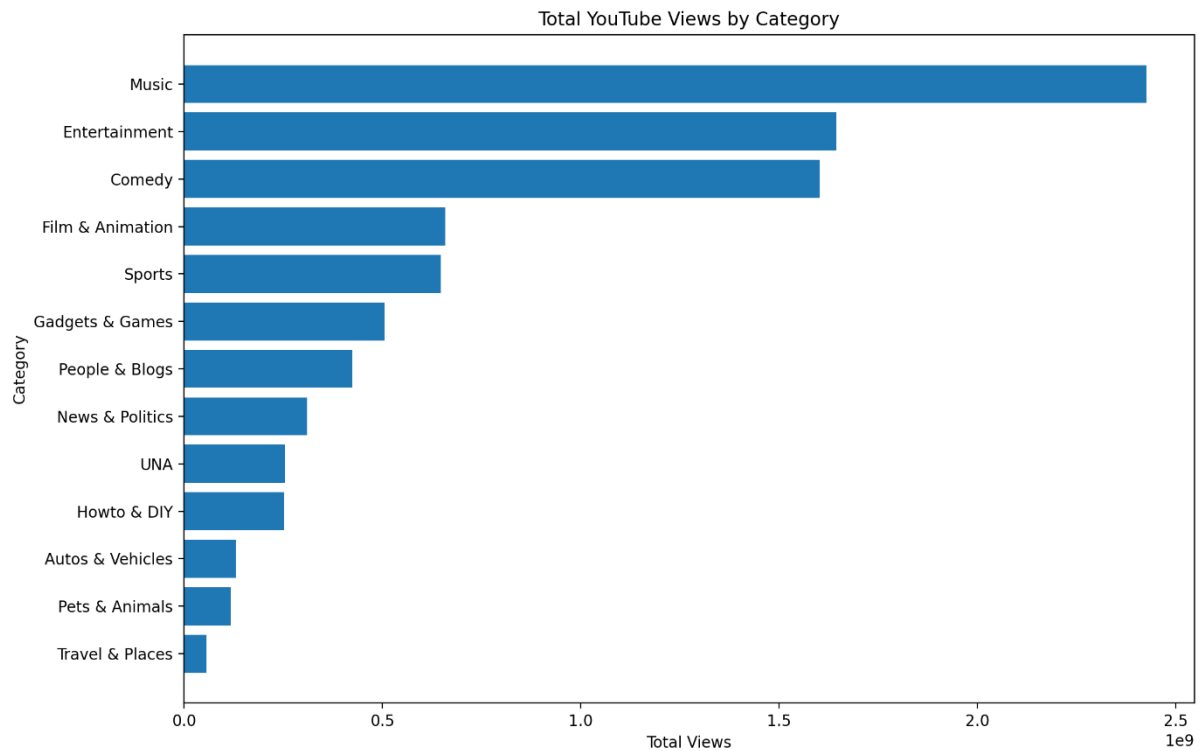


Fig17:category_total_views.png.

Description:

This visualization compares the total number of views received by each category. Music has the highest total view count.

Category-wise Average Rating

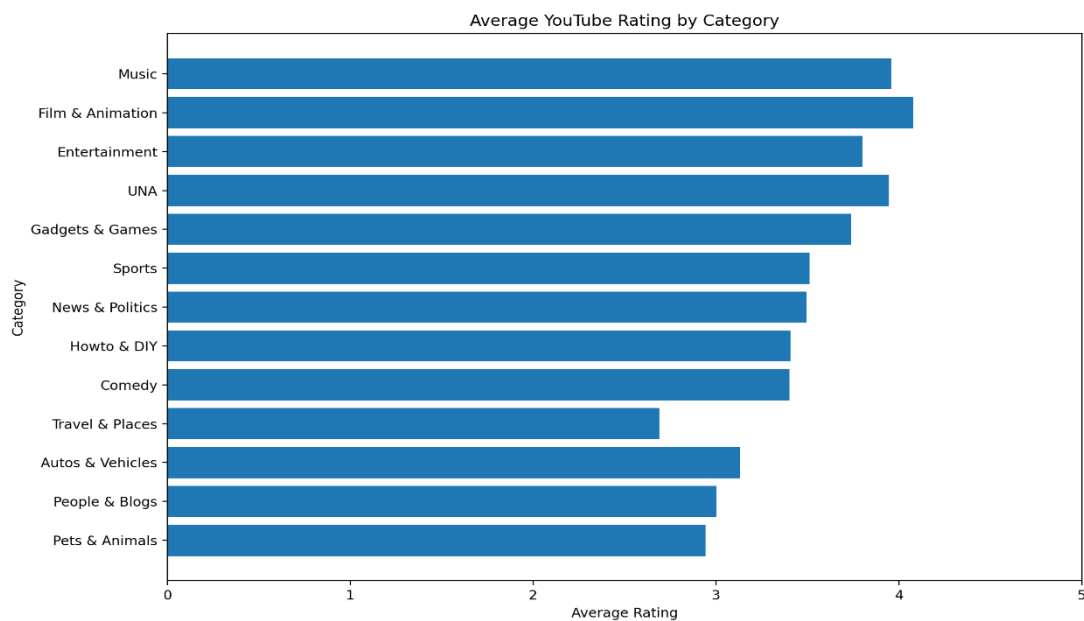


Fig18:category_average_rating.png.

Description:

This visualization compares the average rating of videos across categories. Film & Animation achieves the highest average rating.

7.7 MapReduce and Pig Comparison

Feature	MapReduce	Apache Pig
Language	Java	Pig Latin
Programming Level	Lower-level	Higher-level
Mapper/Reducer	Explicitly implemented	Handled through Pig operators
Grouping	Implemented in Java	GROUP
Counting	Reducer aggregation	COUNT()
Summation	Reducer aggregation	SUM()
Average	Reducer aggregation	AVG()
Code complexity	Higher	Lower
Analytical flexibility	High	High-level and easier
Results	Correct	Consistent with MapReduce

MapReduce provided detailed control over the processing logic, while Pig simplified the implementation of grouping and aggregation operations.

8. Conclusion

This project successfully demonstrated the use of Hadoop HDFS, Java MapReduce, and Apache Pig for analyzing a large YouTube dataset.

A total of 749,361 YouTube records were processed. HDFS was used to store the dataset and organize it into HDFS blocks. The HDFS filesystem was verified successfully, with no missing, corrupt, or under-replicated blocks.

The MapReduce component implemented three major analytical operations:

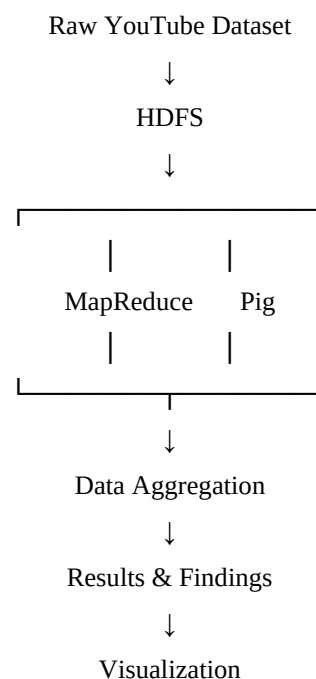
1. Category-wise video count
2. Category-wise total views
3. Category-wise average rating

These operations used the fundamental concepts of mapping, grouping, reducing, and data summarization. The reference criteria identify filtering, data organization, and summarization

as important processing patterns, and the implemented project demonstrates these concepts through category-based aggregation. Apache Pig was then used to perform equivalent analytical operations through Pig Latin. The dataset was loaded from HDFS, grouped by category, and processed using COUNT, SUM, and AVG operations. The results from MapReduce and Pig were consistent. This provides an additional validation of the analytical results. The analysis showed that Music was the most represented category with 179,049 videos and also had the highest total views with 2,426,199,511 views. However, Film & Animation achieved the highest average rating of approximately 4.08.

The project also identified 5,792 records with missing/empty category values, which is an important observation regarding the quality and completeness of the dataset.

Overall, the project demonstrates how Big Data technologies can be used to handle large datasets through a complete workflow:



The project therefore provides practical experience with distributed storage, large-scale data processing, grouping, aggregation, data summarization, and result verification using Hadoop technologies.

The complete implementation, Java MapReduce source code, Pig scripts, HDFS commands, screenshots, and visualizations are available in the GitHub repository:

GitHub:

https://github.com/Imtiaz-1132/BDAL_Final_Project.git